

CSCI 3060U - Software Quality Assurance

Phase 1 Test Plan

Digital Games Distribution System - Front End Requirements Testing Group 15 | Fall 2026

1. Purpose and Scope

This test plan describes how the Phase 1 Front End requirements tests are organized, prepared, and intended to be executed once a Front End implementation is available. Phase 1 focuses on requirements testing only; no Front End program is implemented or executed during this phase.

The test suite contains 131 requirements tests covering valid behaviour, invalid input, boundary conditions, account permissions, session behaviour, Daily Transaction File (DTF) output, and the list_games transaction. The plan also defines how starting-state data, actual outputs, and comparison results will be stored so later runs can be repeated consistently.

2. Test Organization

Tests are grouped by transaction or behaviour category. Each ordinary test has a literal input stream, an expected observable terminal behaviour file, and an exact expected DTF file. Multi-session tests use separate expected DTF files for each session.

Category	Number of Tests
addcredit	11
buy	13
create	11
delete	7
invalid_input	10
list_games	12
login	9

logout	10
refund	15
sell	15
transaction_output	18
Total	131

2.1 File Naming Convention

- <category>_<test number>_input.txt - literal console input, one input line per line.
- <category>_<test number>_expected.txt - expected observable terminal behaviour.
- <category>_<test number>_dtf_expected.txt - exact expected Daily Transaction File content.
- For tests spanning more than one session, session-specific files such as _session1_dtf_expected.txt and _session2_dtf_expected.txt are used.

2.2 Directory Structure

The Phase 1 repository is organized as follows:

```

phase1/
  documents/
    Phase_1_Test_Cases.pdf
    Phase_1_Test_Plan.pdf
    README.txt
  shared_test_data/
    current_users.txt
    available_games.txt
    game_collection.txt
    README.txt
  test_fixtures/
    README.txt
    buy_06/
    buy_07/
    buy_08/
    refund_10/
    refund_11/
    list_games_02/
    list_games_03/
    sell_01/
    sell_02/

```

```
sell_03/  
tests/  
  addcredit/  
  buy/  
  create/  
  delete/  
  invalid_input/  
  list_games/  
  login/  
  logout/  
  refund/  
  sell/  
  transaction_output/  
  README.txt  
scripts/  
  run_tests.bat  
  README.txt  
results/  
  actual/  
  comparisons/  
  README.txt
```

3. Starting-State Test Data

The `shared_test_data` directory contains the default starting state for a test session. These files follow the fixed-width formats defined by the project requirements and the client clarifications used by the group.

- `current_users.txt` - default user accounts, account types, and credit balances.
- `available_games.txt` - default games currently offered for sale.
- `game_collection.txt` - default game ownership information.

Most tests use the shared starting state. When a test requires a different balance, ownership relationship, or set of available games, only the necessary starting-state file is replaced by a test-specific file from `test_fixtures`. All other starting-state files continue to come from `shared_test_data`. This keeps tests independent and reproducible without changing unrelated data.

3.1 Test-Specific Fixture Overrides

buy_06: FullUser starts with 20.00 credit for insufficient-credit testing.
buy_07: FullUser starts with exactly 50.00 credit.
buy_08: FullUser already owns Minecraft.
refund_10: Seller1 starts with exactly 20.00 credit.
refund_11: Seller1 starts with 10.00 credit.
list_games_02: Available Games contains exactly one game.
list_games_03: Available Games contains no games other than END.
sell_01: Minecraft is absent so selling Minecraft is a valid unique-name test.
sell_02: Terraria is absent so selling Terraria is a valid unique-name test.
sell_03: Portal is absent so selling Portal is a valid unique-name test

4. Test Execution Plan

Because Phase 1 does not include a Front End implementation, the tests are not executed during this phase. The scripts directory documents the intended automated process that will be connected to the Front End in a later phase.

1. Select the required test input file and restore the default shared starting-state files.
2. If the test has a fixture override, replace only the required starting-state file with the corresponding file from test_fixtures.
3. Run the Front End with the test input file redirected to standard input.
4. Redirect the terminal output to results/actual using the same category and test number.
5. Save the Daily Transaction File produced by that session to results/actual.
6. For multi-session tests, save each session output separately.
7. Compare actual terminal behaviour with the corresponding *_expected.txt file.
8. Compare the generated DTF with the corresponding *_dtf_expected.txt or session-specific DTF expected file.
9. Save comparison results in results/comparisons and use them to determine pass/fail status.
10. Restore the default starting state before the next independent test.

4.1 Planned Batch Script

The current Phase 1 batch file is a placeholder because the Front End executable does not yet exist. Its purpose is to document the planned automation workflow until the executable can be connected in a later phase.

@echo off

REM Phase 1 planned Front End test runner

REM The Front End executable will be added in a later project phase.

REM Intended process:

REM 1. Run Front End using each *_input.txt file.

REM 2. Redirect terminal output into results\actual.

REM 3. Save the generated Daily Transaction File.

REM 4. Compare actual outputs with expected outputs.

REM 5. Save comparison results in results\comparisons.

echo Phase 1 test runner placeholder.

echo Front End implementation will be connected in a later phase

5. Actual Output and Comparison Organization

The results directory is reserved for outputs generated when the requirements tests are executed against the Front End in later phases.

- results/actual/ stores actual terminal output and actual generated DTF files.
- results/comparisons/ stores the results of comparisons between expected and actual outputs.

Planned actual-output filenames retain the test identifier. For example, buy_01_actual.txt stores terminal output and buy_01_dtf_actual.txt stores the generated DTF. Comparison files use names such as buy_01_output_diff.txt and buy_01_dtf_diff.txt.

A test passes when the observed program behaviour and generated DTF match the expected behaviour defined by the Phase 1 requirements test. Any difference is retained in results/comparisons for investigation and regression testing.

5.1 DTF Comparison

DTF output is compared as exact text because field width, padding, monetary formatting, transaction codes, and record order are requirements being tested. Failed or incomplete transactions are expected to be omitted from the DTF according to the client clarification used by the group. Every valid session ends with the 00 end-of-session record.

6. Requirements Issues and Group Decisions

The project requirements contain several ambiguities or conflicts. The test suite records the following clarifications and deadline assumptions so that expected results are consistent and traceable:

- Where a written field length conflicts with the supplied example, the client-confirmed example length is used.
- Available Games records use the example-defined 49-character record length and a 26-character game-name field.
- Newly created accounts begin with 0.00 credit.
- A seller cannot purchase their own listed game.
- A successful refund removes the corresponding game from the buyer's Game Collection, and the buyer may purchase the game again later.
- A refund is denied if the seller does not have sufficient credit.
- addcredit is denied if the resulting account balance would exceed the maximum allowed balance.
- Failed or incomplete transactions do not create DTF transaction records.
- list_games requires login and is available to all account types. Because no transaction code or record format was provided for list_games before the deadline, the group treats list_games as display-only and does not create a separate DTF transaction record.
- For account deletion, the group assumes that Game Collection entries owned by the deleted user are removed. This is documented as a group design decision because the original requirements do not explicitly define the behaviour.

7. Reproducibility and Maintenance

Each test is intended to be independent. Before a test is run, the known default starting state is restored and any test-specific override is applied. This avoids one test changing the starting conditions of the next test. If later client clarifications change an expected result, the corresponding test-case documentation, expected-output files, fixtures, and this plan should be updated together.

8. Summary

This plan provides a repeatable structure for running the 131 Phase 1 requirements tests once the Front End becomes available. The directory structure separates inputs, expected results, shared data, special fixtures, execution scripts, actual outputs, and comparisons so that failures can be reproduced and later regression runs can use the same requirements tests.